

FoodVision
Final Project Report
Columbia University - M.S. in Applied Analytics

Executive Summary

FoodVision is a computer vision system designed to analyze food images and return meaningful nutritional information. It recognizes dishes, identifies ingredients, and estimates calories through a simple REST API, making it easy to integrate into nutrition apps, health platforms, and food-related tools.

The system runs entirely offline once deployed and is packaged using Docker for easy setup and portability. It stores over 249,000 ingredient embeddings in optimized PyTorch tensors and is built with FastAPI to support fast requests, automatic API documentation, and strong input validation.

This project focuses on applying modern machine learning in a practical, usable way. Instead of requiring custom training, FoodVision uses CLIP's ability to generalize across visual concepts and pairs it with USDA data to return real nutritional insight from raw images.

The architecture is built for reliability and usability. Users interact with the system using straightforward API calls, and responses include food names, ingredient breakdowns, and estimated nutritional values. Because it does not rely on an internet connection, the system is well-suited for mobile use, medical environments, or any setting where connectivity is limited.

Project Overview

FoodVision addresses the growing need for automated nutrition analysis through the integration of computer vision algorithms with practical deployment infrastructure. As nutrition awareness and health management become increasingly important, traditional food logging and calorie counting remain tedious tasks. Entering nutrition information takes time and is often inaccurate since many existing tools rely on private APIs or limited food databases.

This project fills the gap between machine learning models built primarily for research and systems that can be used in real-world production environments. By using CLIP's zero-shot learning capability, FoodVision is able to recognize a wide range of foods without requiring prior training on specific datasets. The system analyzes images using computer vision techniques that identify not only the main dish, but also individual ingredients.

Integration with USDA FoodData Central ensures that calorie estimates are based on authoritative nutrition data maintained by the federal government. This differs from platforms that rely on crowdsourced information, which can be inconsistent or unreliable. FoodVision's three-tier calorie estimation system uses fallback strategies to handle incomplete or unclear inputs.

Key Features:

- Food Recognition: Identifies food items from images using CLIP ViT-B/32 model with confidence scoring

- Ingredient Detection: Fast cosine similarity search across 249,000+ precomputed ingredient embeddings
- Calorie Estimation: 3-layer inference system combining direct matching, ingredient-based calculation, and fuzzy token matching
- Offline Operation: All embeddings stored locally in optimized .pth chunks, no external API calls required
- Production-Ready Deployment: Fully Dockerized with automatic documentation via FastAPI
- RESTful API: Clean endpoints with JSON responses for easy integration

Technical Stack

Framework & Infrastructure:

- FastAPI - Modern Python web framework with automatic OpenAPI documentation
- Uvicorn - High-performance ASGI server
- Docker - Containerization for consistent deployment

Machine Learning:

- PyTorch - Deep learning framework
- OpenCLIP - Open-source implementation of CLIP model

Data Processing:

- NumPy - Numerical computing
- Pillow (PIL) - Image processing
- RapidFuzz - High-performance fuzzy string matching

Data Sources and Processing

USDA FoodData Central

The USDA FoodData Central dataset from the United States Department of Agriculture serves as the nutritional database which FoodVision uses to calculate food calories. The database contains complete nutritional data for thousands of food items which enables our system to calculate exact calorie values. The USDA has maintained its position as the primary organization for food composition data collection since the 1900s while becoming the worldwide leader for nutritional information.

FoodData Central brings together three previous USDA food databases which include Standard Reference and Food and Nutrient Database for Dietary Studies and Branded Food Products Database. The database provides users with access to various nutritional data through a single platform which includes agricultural products and consumer packaged foods. Scientists validated all food categories in the database through multiple testing procedures.

The dataset contains detailed information which extends past the standard calorie tracking data. The current FoodVision calorie estimation system uses USDA data which contains detailed information about macronutrients and micronutrients and vitamins and minerals and other nutritional elements. The database contains full nutritional data which developers can use to create new features that display users with complete nutritional details.

The process of nutritional analysis depends on having the most accurate possible data. The USDA achieves exact results through its implementation of established laboratory methods and

its rigid quality assurance protocols. The food entries contain portion size information and cooking directions along with complete nutritional data. The detailed information in FoodVision allows users to perform exact calculations for complex restaurant dishes and simple home-cooked meals.

Dataset Characteristics:

- Source: USDA FoodData Central (<https://fdc.nal.usda.gov/download-datasets>)
- Coverage: Comprehensive nutritional data for branded foods, survey foods, and foundation foods
- Data Quality: Government-validated nutritional information with standardized serving sizes
- Format: Converted from USDA CSV format to optimized PyTorch .pth tensors

Embedding Generation

The system uses CLIP's dual-encoder structure to establish a common embedding space which enables both image and text data representation. The system enables zero-shot food recognition because it can analyze food categories without needing any specific training data. The model uses the embedding space to link visual data with linguistic information which enables it to recognize relationships between observed images and known language concepts.

The training process of CLIP involved processing 400 million image-text pairs which were collected from internet sources. The model developed its ability to match correct image-text pairs through pre-training while it learned to distinguish between correct and incorrect pairings. The contrastive learning method developed an extensive semantic embedding space which groups similar concepts together regardless of their visual or textual nature.

Food Label Embeddings

The text encoder of CLIP produces precomputed embeddings for food labels by using particular prompts. The text encoder of CLIP generates dense vector representations for food categories through its processing of prompts which begin with 'a photo of [food_name]'. The specific design of prompts proves essential because CLIP received its training data from image captions which followed similar linguistic structures.

The system identifies objects through its prompt selection process structure. The model achieves better performance in food item recognition because we use 'a photo of' as a prefix which matches the training language patterns of CLIP. The model achieves improved food item recognition through this basic method which enhances its ability to understand image content.

Ingredient Embeddings

The ingredient recognition system relies on a massive library of 249,000+ precomputed embeddings. Each embedding represents a specific ingredient or food component extracted from the USDA database. This comprehensive coverage ensures that FoodVision can identify ingredients ranging from common items like tomatoes and cheese to more specialized components like specific spices and food additives.

The scale of the ingredient embeddings presents both opportunities and challenges. With such extensive coverage, the system can perform fine-grained ingredient analysis, detecting subtle components that contribute to a dish's nutritional profile. However, managing and searching through this large embedding space requires careful optimization to maintain fast inference speeds.

- Scale: 249,000+ ingredient embeddings
- Storage: Distributed across 13 chunks of approximately 40MB each
- Optimization: Stored as PyTorch tensors for fast GPU/CPU operations
- Search Method: Cosine similarity between image embeddings and ingredient embeddings

Data Pipeline Architecture

Preprocessing Phase:

- USDA data downloaded and parsed from CSV format
- Food names and ingredients extracted and normalized
- CLIP model loaded to generate text embeddings
- Embeddings computed in batches and stored as PyTorch tensors
- Data split into 13 chunks for efficient loading and memory management

Runtime Processing:

- Image received via API endpoint
- Image preprocessed using CLIP's standard transformations
- Image embedding generated via CLIP vision encoder
- Similarity computed against preloaded embeddings
- Results ranked and returned as JSON response

Model Architecture and Implementation

CLIP (Contrastive Language-Image Pretraining)

The field of computer vision underwent a significant change through CLIP because it teaches visual concepts by processing natural language instructions. The model learns to link images with text descriptions through CLIP which enables it to perform tasks without needing pre-defined categories in labeled datasets. The method creates a new approach for training computer vision models.

The operation of computer vision models depends on large databases which contain images that have received specific labels for their particular tasks. A model needs 1000 labeled examples for each food category to achieve food recognition. The process of gathering data proves costly and takes a long time while preventing the model from learning new categories without complete retraining. The model learns from natural language data to eliminate its requirements for specific training data.

The core principle of CLIP depends on the fact that web images frequently receive descriptive text annotations. The model learns to recognize visual concepts through linguistic descriptions which it receives from image-text pairs during training. The model achieves successful identification of unknown objects and scenes through its ability to understand textual descriptions of these items.

The FoodVision system enables users to identify various food items from diverse cuisines through its revolutionary technology which operates independently of food labeling information. The system identifies Japanese sushi and Italian pasta and Indian curry through its language-based understanding of these concepts which allows it to recognize them from text descriptions alone.

Model Specifications:

- Architecture: Vision Transformer (ViT-B/32)
- Image Encoder: Vision Transformer with 12 layers, 768 hidden dimensions
- Text Encoder: Transformer with masked self-attention
- Embedding Dimension: 512-dimensional vectors
- Training Data: Pre-trained on 400 million image-text pairs from the internet

Vision Transformer Architecture

The Vision Transformer (ViT) processes images by dividing them into fixed-size patches, treating each patch as a token similar to words in natural language processing. This approach represents a departure from traditional convolutional neural networks that process images through hierarchical feature extraction. Instead, ViT applies the transformer architecture, originally designed for text, directly to image patches.

The transformer architecture has dominated natural language processing since its introduction in 2017. Its self-attention mechanism allows the model to weigh the importance of different parts of the input when processing each element. By treating image patches as tokens, Vision Transformers bring this powerful mechanism to computer vision, enabling the model to capture long-range dependencies across the entire image.

Traditional convolutional networks process images through local receptive fields that gradually expand through deeper layers. While effective, this approach can struggle to capture relationships between distant regions of an image. In contrast, ViT's self-attention mechanism can directly model relationships between any two patches, regardless of their spatial distance. This global context awareness is particularly valuable for food recognition, where understanding the overall composition of a dish can be as important as identifying individual components.

Processing Pipeline:

- Patch Extraction: Input image (224x224) divided into 16x16 patches, creating 256 patches
- Linear Projection: Each patch flattened and projected to 768 dimensions
- Positional Encoding: Learnable position embeddings added to preserve spatial information
- Transformer Encoding: 12 layers of multi-head self-attention and feed-forward networks
- Output Projection: Final layer produces 512-dimensional embedding

Zero-Shot Classification

FoodVision leverages CLIP's zero-shot capabilities, meaning the model can recognize food categories it was never explicitly trained on. This is achieved through the shared embedding space where similar concepts cluster together, allowing the model to generalize from its pre-training.

Inference Process:

- Image embedding generated by passing through CLIP vision encoder
- Cosine similarity computed between image embedding and all precomputed food label embeddings
- Similarities converted to probabilities via softmax function
- Top-k predictions returned with confidence scores

Calorie Estimation System

The calorie estimation system implements a 3-layer fallback strategy to maximize accuracy while maintaining robustness. This hierarchical approach tries the most specific matching first, then falls back to ingredient-based estimation, and finally uses fuzzy matching for edge cases.

- Layer 1: Direct Food Matching
 - Attempts exact string match between predicted food name and USDA database entries. This provides the most accurate calorie estimate when successful, as it uses the exact nutritional data for the identified food item.
- Layer 2: Ingredient-Based Estimation
 - If direct matching fails, the system identifies individual ingredients using cosine similarity search across the 249,000+ ingredient embeddings. Calorie values are aggregated from matched ingredients, weighted by their similarity scores.
- Layer 3: Fuzzy Token Matching
 - As a final fallback, RapidFuzz library performs fuzzy string matching over normalized calorie database keys. This catches variations in naming conventions, misspellings, and partial matches that the previous layers might miss.

Application Architecture

FoodVision uses FastAPI to create a REST API. The main endpoint is POST /analyze, which accepts food images as multipart/form-data and returns JSON responses with the predicted food name, CLIP confidence score (0-1), detected ingredients with similarity scores, calorie estimation method (direct/ingredient/fuzzy), and estimated calories. The application is structured in four layers: the API layer handles HTTP requests and responses through FastAPI routes, the service layer processes images and estimates calories, the model layer interfaces with the CLIP model and computes embeddings, and the data layer stores precomputed PyTorch embeddings. The complete application is packaged in a Docker container with all dependencies, model weights, and embeddings.

The application is fully containerized using Docker, ensuring consistent behavior across different deployment environments. The Docker image includes all dependencies, model weights, and precomputed embeddings, making deployment straightforward.

Results and Evaluation

Model Performance

The CLIP system achieved top results in food item identification through zero-shot learning because it did not need any food-specific training data. The system successfully identified various food items including pizzas and sandwiches and fruits and prepared meals. The system achieved its best results when food items appeared in plain view with proper lighting and positioned at the center of the frame. The system produced superior recognition results for foods

which had distinct visual characteristics but struggled with foods that had ambiguous visual appearance including soups and mixed dishes. The system generated confidence ratings which corresponded to both the image quality and their level of clarity.

Recognition Capabilities

The system identifies food at three different levels which include specific ingredients (tomatoes and cheese and flour) and complete meals (pizza and burgers) and general food groups. The system produces confidence ratings which show the precision level of its predicted outcomes. The system performs cosine similarity searches across 249,000+ precomputed embeddings to identify individual components in complicated meals.

Calorie Estimation Performance

The 3-layer estimation system followed the design specifications which its developers had originally created. The system used direct USDA matching to locate foods which had matching characteristics. The system used ingredient-based aggregation to calculate nutritional values by adding up the detected components of complex dishes. The system used fuzzy string matching to connect predicted labels with database entries by matching names that varied slightly. The system used multiple layers to minimize search errors when it generated nutrition information for all user input entries.

System Performance

The system processed data at high speed because it stored precomputed embeddings in PyTorch tensors which were optimized for performance. The 13-chunk storage design (approximately 40MB each) allowed for efficient memory usage and fast loading times. The system operated at high speed because it operated offline which eliminated network-related delays. The Docker container maintained a constant system level throughout all deployment environments.

Strengths and Limitations

The system functions independently through its precomputed embeddings system which provides quick inference and supports 249,000+ ingredients with USDA-backed nutritional data and allows Docker deployment for portability. The system fails to calculate portion sizes because it uses standard serving amounts for calorie calculations and it performs poorly with complicated multi-ingredient dishes and it requires clear images for optimal performance and it may show biases toward foods that appear frequently in CLIP's training data.

Technical Challenges and Solutions

Embedding Storage

The system experienced a critical problem when trying to access more than 249,000 embeddings during inference because it caused memory overload and performance deterioration. The solution needed an optimized file format to store embeddings in chunked files which enhanced both file loading speed and computational performance. The system runs extensive data processing operations while keeping its memory usage at its most efficient level.

Offline Operation

The system needed to function independently from the internet to access model weights and nutritional data. The solution needed all required resources to be directly incorporated into the

application environment. The system operates independently from external services because of its design which provides reliable performance in environments with limited or no network access.

Label Cleaning

The model produced output labels that contained training-related data which made them difficult for users to understand. The solution to this problem required adding a post-processing step which removed unneeded text from the generated output. The system produces output that users can understand better because of this improvement.

Calorie Estimation Accuracy

The system failed to match predicted food labels with nutritional data because it encountered different naming conventions and complicated food preparation methods. The system uses a layered matching system which starts with direct matches before using ingredient-based logic and approximate matching methods when direct matches are unavailable. The system maintains high reliability because it uses multiple matching methods which continue to function when it cannot find exact matches.

Conclusion

The FoodVision system shows how computer vision models work in real-world applications through its complete operational system that functions as a production-ready solution. The project achieves high food recognition accuracy through OpenAI's CLIP model which performs zero-shot learning and the 3-layer calorie estimation system provides exact nutritional analysis from USDA data.

The system design achieves high performance because of its architecture which makes deployment easy. The system achieves quick processing times because it uses precomputed embeddings which are stored in PyTorch tensors that operate efficiently. The system uses its 249,000+ ingredient embeddings and USDA nutritional data to analyze different food items and provide detailed nutritional information.

The development process required solutions for multiple technical obstacles which became essential for deploying machine learning models in operational environments. The system needed optimized storage formats together with loading methods to efficiently process its extensive embedding database. The 3-layer calorie estimation system reflects a pragmatic approach to handling uncertainty, implementing fallback strategies that maintain system reliability even when perfect matches are unavailable.

FoodVision successfully bridges cutting-edge machine learning research and practical production deployment, demonstrating that advanced computer vision can be made accessible, efficient, and reliable for real-world nutritional analysis applications.